

Google Colab 기초

클라우드 기반 Jupyter 노트북 환경

Google 계정만으로 무료 GPU를 활용한 딥러닝 실습 환경 구축

LLM 과정 Day 1

목차

01 학습 목표

- 인터페이스 이해
- 셀 실행 방법
- GPU 설정
- 패키지 관리

02 Colab 기본 사용법

- 노트북 생성
- 셀의 종류
- 단축키
- 실행 순서

03 T4 GPU 설정

- 런타임 설정
- GPU 환경 확인
- T4 연결
- 설정 주의사항

04 메모리 제약

- VRAM 15GB
- 양자화 기술
- OOM 대처
- 실습 가능 범위

05 세션 관리

- 세션 초기화
- Drive 저장
- 폴더 구조
- 저장 습관

06 패키지 설치

- pip install
- import 패턴
- 주요 패키지
- 에러 해결

학습 목표

인터페이스

- Google Colab의 기본 인터페이스 이해
 - 셀 실행 방법 습득
 - 코드 셀과 텍스트 셀 구분

GPU 설정

- 런타임 유형을 T4 GPU로 설정
 - 하드웨어 가속기 선택
 - 런타임 재시작 이해

환경 확인

- GPU 환경 코드로 확인
 - GPU 이름 확인 (Tesla T4)
 - VRAM 메모리 크기 확인 (~15GB)

패키지 관리

- !pip install 명령 사용
- 패키지 설치 및 import
- 필요 시점에 설치하는 습관

세션 관리

- 세션 초기화 문제 이해
- Drive 저장 습관 갖추기
- 중요 결과물 보존 방법

Google Colab이란?

클라우드 환경

- 브라우저 기반 클라우드 Jupyter 노트북 환경
- 별도 설치 없이 즉시 사용 가능
- Google 계정만으로 접근
- 딥러닝 실습에 최적화된 환경

접속 및 실행

- 접속:
colab.research.google.com
- 실행 단위: 셀 (Cell)
- 코드 셀: 파이썬 코드 실행
- 텍스트 셀: 마크다운 형식 메모

GPU 지원

- 무료 GPU: Tesla T4
- VRAM 약 15GB 제공
- 유료 플랜: Colab Pro (A100 등)
- 딥러닝 모델 학습 가능

저장 방식

- Google Drive 자동 연동
- 노트북 파일 자동 저장
- 결과물 Drive 저장 권장
- 세션 종료 시 파일 보존

노트북 생성 및 셀의 종류

노트북 생성 방법

- 방법 1: colab.research.google.com 접속 → 새 노트 클릭
- 방법 2: Google Drive → 새로 만들기 → 더보기 → Google Colaboratory
- 방법 2 권장: Drive 특정 폴더에 바로 저장되어 정리 편리

코드 셀

- 파이썬 코드 작성 및 실행
- 실행 방법: Shift + Enter 또는 셀 왼쪽 ▶ 버튼
- 코드 실행 결과 즉시 확인
- 변수 선언 및 함수 정의

텍스트 셀

- 설명 및 메모 작성 (마크다운 형식)
- 더블클릭으로 편집 모드 진입
- Shift + Enter로 렌더링
- 문서화 및 주석 작성에 활용

셀 실행 순서의 중요성

- 셀을 위에서 아래로 순서대로 실행 필수
- 중간 셀부터 실행 시 에러 발생 가능
- 세션 재시작 후: '런타임 → 모두 실행 (Ctrl + F9)'
- 변수 및 패키지 순차적 로드 필요

자주 쓰는 단축키

Shift + Enter

- 현재 셀 실행 후 다음 셀로 이동
- 가장 많이 사용하는 단축키
- 순차적 실행에 최적화

Ctrl + Enter

- 현재 셀 실행 (이동 안 함)
- 같은 셀 반복 실행 시 유용
- 결과 확인 후 수정 작업

Ctrl + M + B

- 아래에 새 코드 셀 추가
- 빠른 셀 생성
- 작업 흐름 유지

Ctrl + S

- 노트북 저장
- 주기적 저장 습관 필요
- 작업 내용 보존

Ctrl + /

- 선택 영역 주석 처리
- 코드 임시 비활성화
- 디버깅 시 유용

T4 GPU 런타임 설정 방법

1단계: 런타임 메뉴 접근

- 상단 메뉴에서 '런타임(Runtime)' 클릭
- 드롭다운 메뉴가 나타남
- 여러 옵션 중 '런타임 유형 변경' 선택

2단계: 런타임 유형 변경

- 'Change runtime type' 대화상자 열림
- 하드웨어 가속기 옵션 확인
- 기본값은 CPU로 설정되어 있음

3단계: T4 GPU 선택

- 하드웨어 가속기 드롭다운 클릭
- 'T4 GPU' 옵션 선택
- GPU 사용 설정 완료

4단계: 저장 및 재시작

- '저장(Save)' 버튼 클릭
- 런타임이 자동으로 재시작됨
- ⚠ 주의: 노트북 열자마자 가장 먼저 설정

GPU 환경 확인

GPU 연결 여부

- 코드로 GPU 연결 확인
 - 정상 출력값: True
- GPU가 정상적으로 연결되었음을 의미
- 첫 번째 확인 항목

GPU 이름

- 연결된 GPU 모델명 확인
 - 정상 출력값: Tesla T4
- Colab 무료 버전 기본 GPU
- 두 번째 확인 항목

VRAM 크기

- 사용 가능한 메모리 확인
 - 정상 출력값: ~15 GB
- 모델 로드 가능 여부 판단 기준
- 세 번째 확인 항목

False 출력 시 조치

- 런타임 유형 재설정 필요
- 런타임 → 모두 재시작 실행
- GPU 할당량 초과 시 대기
- 다음 날 재시도 권장

T4 메모리 제약과 실습 가능 범위

T4 GPU 기본 사양

- VRAM 용량: 약 15GB
- Colab 무료 버전 제공
- 모델 크기와 양자화 방식에 따라 실습 가능 여부 결정

2-3B 모델 실습

- 4-bit 양자화 적용 시: 약 2-3GB 사용
- 매우 안정적인 실습 환경
- 이번 과정에서 주로 사용하는 모델 크기

7B 모델 실습

- 4-bit 양자화 적용 시: 약 5-6GB 사용
- T4에서 실습 가능
- FP16 사용 시 약 14GB로 불안정

OOM 에러 대처법

- OutOfMemoryError 발생 시 조치
- batch_size 줄이기 (4→2→1)
- 불필요한 변수 삭제
- 런타임 재시작 후 처음부터 재실행

세션 초기화 문제와 Drive 저장 습관

세션 종료 시점

- 비활성 약 90분 후 자동 종료
- 연결 끊김 시 세션 초기화
- 무료 플랜 최대 세션: 약 12시간

초기화되는 항목

- 설치한 패키지 (pip install)
 - 선언한 변수·함수
- /content/ 내부 생성 파일
- 모두 사라지므로 주의 필요

유지되는 항목

- 작성한 코드 (노트북 셀)
- Google Drive 저장 파일
- 세션 재시작 후에도 유지됨

Drive 저장 필수

- 중요한 결과물은 반드시 Drive 저장
 - 점심 전 저장 습관 필수
- 세션 종료 대비 필수 조치

권장 폴더 구조

- 내 드라이브/llm_course/
 - day1, day2, day3, day4, day5
- 날짜별 폴더로 체계적 관리

패키지 설치 패턴

!pip install 명령

- 터미널 명령으로 패키지 설치
 - 셀 앞에 ! 붙여서 실행
 - 시스템 명령 실행을 의미
 - 기본 설치 명령어

-q 옵션

- 설치 로그 출력 억제
- quiet 모드로 실행
- 화면이 깔끔해짐
- 선택적 사용 옵션

import 명령

- 설치된 패키지를 세션으로 불러 오기
- 파이썬 코드에서 사용 가능하게 함
- 설치 후 반드시 실행
- 버전 확인도 가능

주요 패키지 목록

- openai: API 호출 (Day 1)
- datasets: 데이터셋 로드 (Day 3)
- transformers: 모델 로드 (Day 3-4)
- peft, trl: 파인튜닝 (Day 4)
- gradio: 웹 배포 (Day 5)
- 필요한 날에 그때그때 설치

흔한 에러와 해결법

ModuleNotF

- 패키지 미설치 또는 세션 초기화
- `!pip install -q 패키지명` 실행
- `import` 후 재시도

NameError

- 변수 정의 셀 미실행
- 위쪽 셀부터 순서대로 재실행
- 런타임 → 모두 실행 권장

CUDA OOM

- GPU 메모리 부족 에러
- `batch_size` 줄이기
- 불필요한 변수 삭제
- 런타임 재시작

ConnectionError

- API 호출 실패 (네트워크 문제)
- 잠시 후 재시도
- `try/except` 패턴 사용

JSONDecodeError

- LLM 응답이 JSON 형식 아님
- 프롬프트에 JSON 출력 강조
- 파싱 전처리 추가